Blue/Green Deployment on AWS

Canary Traffic Shift using an Application Load Balancer

Project	Blue/Green Deployment with Canary Strategy
Platform	AWS (KodeKloud Labs) - Region us-east-1
Services	VPC, EC2, Target Groups, Application Load Balancer
Date	07 October 2026

1. Project Overview

In this project I deployed two identical, isolated environments - **Blue (Version 1)** and **Green (Version 2)** - behind one AWS Application Load Balancer (ALB). The ALB is the single entry point for users. By changing the **weights** of the listener rule, traffic was moved from Blue to Green step by step, without any downtime.

Why Blue/Green + Canary?

- **Zero downtime** - the old version keeps running while the new one is tested.
- **Low risk** - only a small share of users (20%) see the new version first.
- **Easy rollback** - if Green has errors, set the weights back to Blue 100.
- **Full cutover** - when Green is healthy, send 100% traffic to it.

Architecture

Component	Name	Details
VPC	BG-VPC	10.0.0.0/16
Public subnet 1	Public-1	us-east-1a, 10.0.1.0/24
Public subnet 2	Public-2	us-east-1b, 10.0.2.0/24
Internet Gateway	BG-IGW	Attached to BG-VPC
Route table	BG-RT	0.0.0.0/0 to BG-IGW; both subnets associated
Security group	Web-SG	Inbound HTTP 80 from 0.0.0.0/0
EC2 (Blue)	Blue-Server	t3.micro, Amazon Linux 2023, Public-1
EC2 (Green)	Green-Server	t3.micro, Amazon Linux 2023, Public-2
Target groups	TG-Blue, TG-Green	HTTP 80, one server each
Load balancer	BlueGreen-ALB	Internet-facing, HTTP:80 listener

Traffic flow: **User** → **ALB (HTTP:80)** → **weighted rule** → **TG-Blue (Blue-Server) / TG-Green (Green-Server)**

2. Task 1 - Networking

An ALB needs at least two public subnets in different Availability Zones, so I built the network first.

Step 1: Create the VPC

1. Open **VPC** → **Create VPC**.
2. Name: **BG-VPC**, IPv4 CIDR: **10.0.0.0/16**.
3. Click **Create VPC**.

Step 2: Create two public subnets

Name	Availability Zone	CIDR
Public-1	us-east-1a	10.0.1.0/24
Public-2	us-east-1b	10.0.2.0/24

Step 3: Enable auto-assign public IPv4

1. Select **Public-1** → Actions → Edit subnet settings.
2. Tick **Enable auto-assign public IPv4 address** and Save.
3. Repeat for **Public-2**.

Step 4: Internet Gateway and routes

1. Create Internet Gateway **BG-IGW** and attach it to **BG-VPC**.
2. Create route table **BG-RT** in BG-VPC.
3. Add route **0.0.0.0/0** → target **BG-IGW**.
4. Associate **Public-1** and **Public-2** with BG-RT.

Result: A custom VPC with two public subnets that have internet access.

3. Task 2 - Blue Environment (Version 1)

The first server represents the current live version of the application.

1. EC2 → **Launch instances**. Name: **Blue-Server**.
2. AMI: **Amazon Linux 2023**, type: **t3.micro**, no key pair.
3. Network: VPC **BG-VPC**, subnet **Public-1**.
4. Create security group **Web-SG** with inbound **HTTP (80)** from 0.0.0.0/0.
5. In **Advanced details** → **User data**, paste the script below, then launch.

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo '<html><body style="background-color:blue; color:white"><h1>VERSION 1
(BLUE)</h1></body></html>' > /var/www/html/index.html
```

The user-data script installs Apache (httpd), starts it, enables it on boot and writes a blue page as index.html.

- Instance is in **Public-1**: Yes.
- Web-SG allows port 80: Yes.

4. Task 3 - Green Environment (Version 2)

The second server represents the new version that we want to test.

1. EC2 → **Launch instances**. Name: **Green-Server**.
2. Same AMI and type (Amazon Linux 2023, t3.micro), no key pair.
3. Network: VPC **BG-VPC**, subnet **Public-2**.
4. Security group: **Select existing** → **Web-SG** (untick the default one).
5. User data: same script, but with a green page:

```
echo '<html><body style="background-color:green; color:white"><h1>VERSION 2
(GREEN)</h1></body></html>' > /var/www/html/index.html
```

5. Task 4 - Target Groups

Target groups tell the load balancer where to send traffic. One target group is created per environment.

1. EC2 → **Target Groups** → **Create target group**.
2. Type: **Instances**, name: **TG-Blue**, VPC: **BG-VPC**, protocol **HTTP:80**.
3. Register **Blue-Server** (Include as pending below) and create.
4. Repeat: name **TG-Green**, register **Green-Server**.

Target group	Registered target
TG-Blue	Blue-Server
TG-Green	Green-Server

6. Task 5 - Application Load Balancer

The ALB is the single entry point for users and decides which target group receives each request.

1. EC2 → **Load Balancers** → **Create load balancer** → **Application Load Balancer**.
2. Name: **BlueGreen-ALB**.
3. Network mapping: VPC **BG-VPC**, tick **us-east-1a (Public-1)** and **us-east-1b (Public-2)**.
4. Security group: remove default, select **Web-SG**.
5. Listener **HTTP:80** → default action: forward to **TG-Blue**.
6. Create and wait until the state is **Active**.

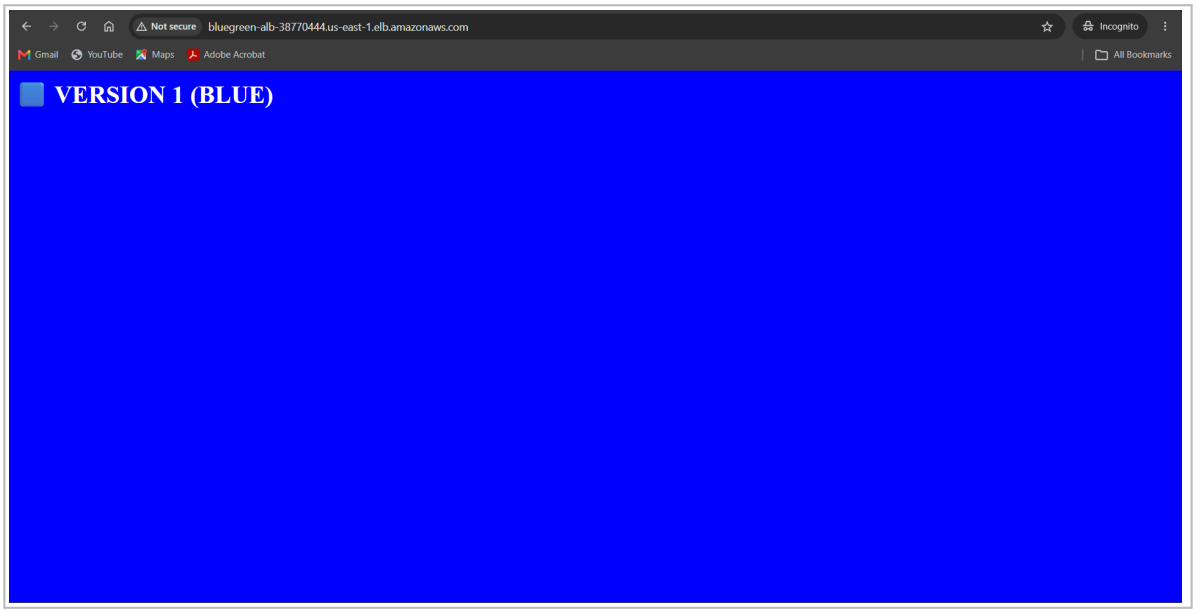


Figure 1: Opening the ALB DNS name in the browser - the default rule forwards to TG-Blue, so Version 1 (Blue) is shown.

7. Task 6 - Traffic Shift (Canary Deployment)

Step 1: Verify the status quo (100% Blue)

I copied the ALB DNS name, opened it with **http://** and refreshed several times. The page always stayed Blue.

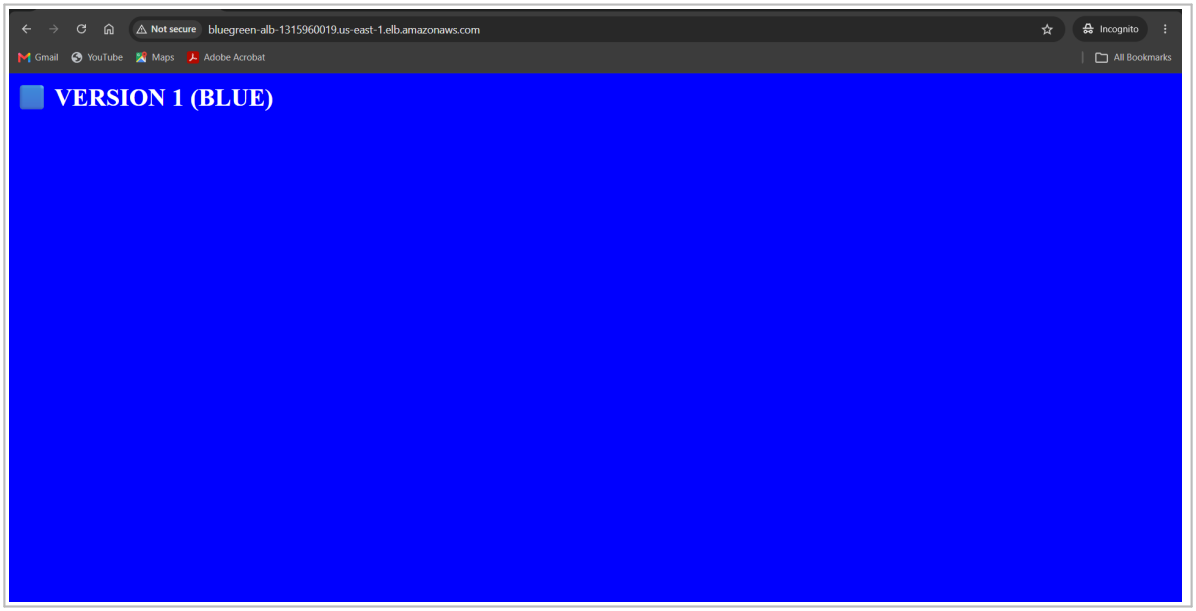


Figure 2: Before the shift - every refresh shows VERSION 1 (BLUE).

Step 2: Edit the listener rule (80 / 20)

1. Load Balancer → **Listeners and rules** → select **HTTP:80** → Manage rules → Edit rule.
2. Under Routing actions set weights: **TG-Blue = 80**, **TG-Green = 20**.
3. Click **Save changes**.

Step 3: Wait for propagation

I waited 30 to 60 seconds so the new rule could be applied.

Step 4: Test the shift

After refreshing many times, most responses were Blue, and about 1 in 5 was Green - this matches the 80/20 weights.

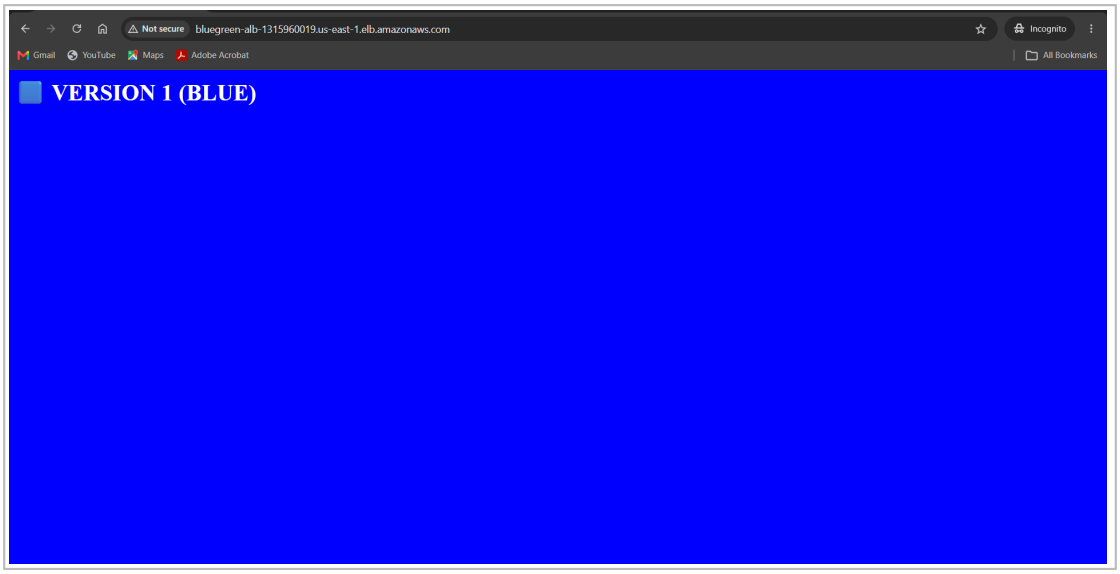


Figure 3: During the canary - most requests still return VERSION 1 (BLUE).

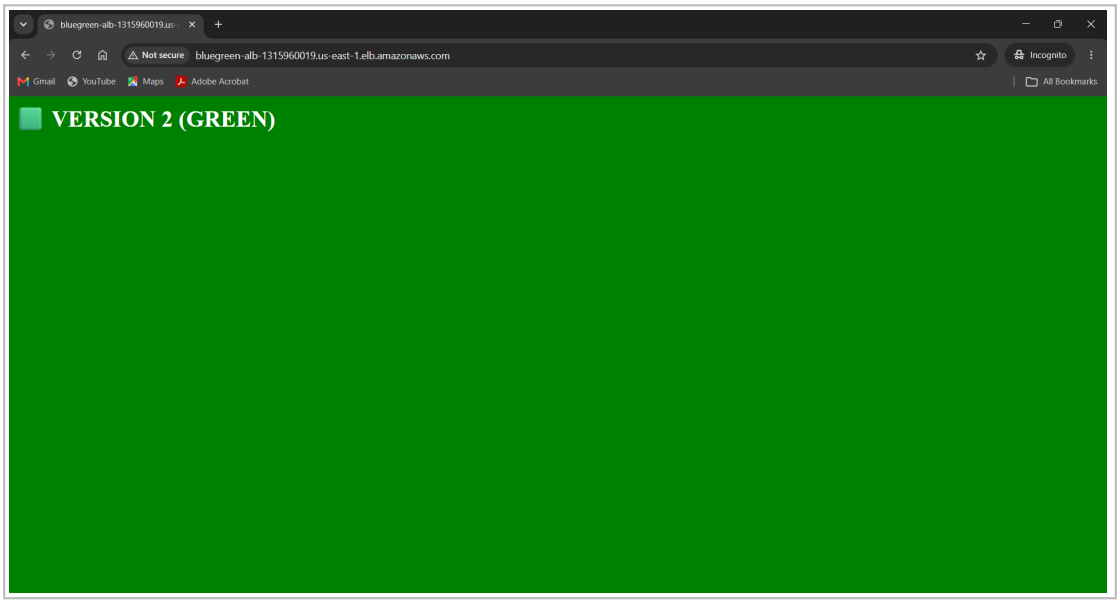


Figure 4: During the canary - some requests return VERSION 2 (GREEN) from TG-Green.

Step 5: Full cutover (100% Green)

1. Edit the rule again.
2. Set weights: **TG-Blue = 0**, **TG-Green = 100**.
3. Click **Save changes**.

Result: All users now see VERSION 2 (GREEN). The Blue server is still running, so a rollback only needs the weights changed back.

8. Weight Summary

Stage	TG-Blue	TG-Green	User sees
Initial	100	0	Always Blue
Canary	80	20	Mostly Blue, sometimes Green
Full cutover	0	100	Always Green
Rollback (if needed)	100	0	Always Blue

9. Skills Learned

- **VPC networking:** multi-AZ public subnets, IGW and route tables.
- **EC2 provisioning:** automatic web server setup with user data.
- **Target groups:** separating environments for traffic control.
- **Application Load Balancer:** listeners and weighted routing rules.
- **Canary deployment:** safely releasing a new version to a small share of users first.

10. Conclusion

This project showed how a new application version can be released with zero downtime. Using two environments behind one ALB and shifting traffic with weighted rules reduces risk and makes rollback quick and simple.